

SDF Project Report (CS1023)

MiniML Abstract Machine with Exception Handling

Akshat Banzal Roll Number: CS24BTECH11005

Atul Boyal Roll Number: CS24BTECH11008

2 May 2025

1 Overview

This project extends the MiniML language with support for exceptions. Specifically, we added two new exception mechanisms:

- `DivisionByZero`
- `GenericException`

We modified the abstract machine implementation in `machine.ml` to support raising and catching exceptions using the `IRaise` and `ITry` instructions.

2 Machine Architecture

The abstract machine executes MiniML programs using:

- A **stack of frames** representing program code.
- A **stack of values** for evaluation.
- A **stack of environments** for variable bindings.

A machine value (`mvalue`) can be:

- An integer (`MInt`)
- A boolean (`MBool`)
- A function closure (`MClosure`)

3 Instruction Set

There are a variety of instructions. We implemented `IRaise` and `ITry`

Arithmetic Instructions

<code>IDiv</code> , <code>IMult</code> , <code>IAdd</code> , <code>ISub</code>
--

Comparison Instructions

<code>IEqual</code> , <code>ILess</code>
--

Stack Manipulations

```
IInt of int
IBool of bool
IVar of name
IClosure of name * name * frame
```

Control Flow

```
IBranch of frame * frame
ICall
IPopEnv
```

Exception Handling

```
IRaise of string
ITry of frame * (name * frame) list
```

4 Exception Handling Semantics

4.1 IRaise

This instruction immediately raises a named exception. For example:

```
| IRaise exn_name -> raise (Machine_error exn_name)
```

4.2 ITry

This instruction implements `try ... with ...` by:

1. Executing the `try` block.
2. Catching exceptions and matching them to handlers.
3. Executing the matched handler block if found.

5 Division By Zero

Division is now guarded against divide-by-zero errors using:

```
| IDiv ->
    try (frms, div stck, envs)
    with Failure _ -> raise (Machine_error "DivisionByZero")
```

6 Sample Example

An example demonstrating exception handling:

```
try
  let x = 10 / 0 in x
with
  DivisionByZero -> 0
```

This will produce the result 0 due to catching the divide-by-zero.

7 Testing Strategy

We tested the machine using:

- Arithmetic operations
- Boolean comparisons
- Function application
- Closures and environments
- Exception handling via `IRaise` and `ITry`

8 Limitations

- Only string-tagged exceptions are supported.
- Stack unwinding is minimal; no backtrace or nested `try` evaluation.

9 Build Instructions

1. Install Dune and OCaml.
2. Navigate to the `src/miniml` directory.
3. Run: `dune build`
4. Use the provided Python CLI to run the machine.

10 Git Repository

The project is maintained on GitHub:

- Tags were created for each milestone: `syntax`, `machine`, `testing`.
- Code is modular and documented.

11 Learnings

- Understood the working of abstract machines.
- Learned to extend a compiler backend and runtime.
- Gained experience with OCaml and functional programming.